

Lecture 11

Type Inference

Type-checking

- (Static) **type-checking** can reject a program before it runs to prevent the possibility of some errors
 - A feature of **statically typed languages**
- Dynamically typed languages do little such checking
 - So might try to treat a number as a function at run-time
- OCaml (and Java, C#, Scala, C, C++) statically typed
 - Every binding has 1 type, determined **before** program runs

Implicitly typed

- OCaml is statically typed
- OCaml is **implicitly typed**: rarely need to write down types
 - Still statically typed: Much more like Java than Javascript!

```
let f x = (* infer val f : int -> int *)  
    if x > 3 then 42 else x * 2
```

```
let g x = (* report type error *)  
    if x > 3 then true else x * 2
```

Type inference

- **Type inference problem:** Give every binding/expression a type such that type-checking succeeds
 - Fail if and only if no solution exists
- In principle, could be a pass before the type-checker
 - In practice, often implemented together
- Type inference can be easy, difficult, or *impossible*
 - Depends on language details
 - Subtle, elegant, and *not magic*: OCaml

Overview

- Will describe the OCaml type inference algorithm via several examples
 - General algorithm is a slightly more advanced topic
 - Supporting nested functions also a bit more advanced/tricky
- Enough to help you “do type inference in your head”
 - And appreciate that it’s not magic

Key steps

- Determine types of bindings in order
 - So you cannot use later bindings: will not type-check
- For each `let` binding:
 - Analyze definition to build a set of *constraints*
 - Example: If see `x > 0`, then `x` must have type `int`
 - Example: If see `if true then y else z`, then `y` and `z` must have same type
 - Type error if no way for all facts to hold (over-constrained)
- Afterward, use type variables (e.g., `'a`) for any unconstrained types
 - Example: An unused argument can have any type
- (Finally, enforce the *value restriction*, discussed later)

First example

After this example, will go much more step-by-step

- Like the automated algorithm does

```
let x = 42 (* val x : int *)  
  
let f y z w =  
  if y (* y must be bool *)  
  then z+x (* z must be int *)  
  else 0 (* both branches have same type *)  
  
(* f must return an int  
   f must take a bool * int * ANYTHING  
   so val f : bool -> int -> 'a -> int  
   *)
```

Relation to Polymorphism

- Central feature of OCaml type inference: it can infer types with type variables
 - Great for code reuse and understanding functions
- But remember these are orthogonal concepts
 - Languages can have type inference without type variables
 - Languages can have type variables without type inference

Key Idea

- Collect all the constraints needed for type-checking
- Solve constraints
 - 0 solutions is a type error
 - Multiple solutions is polymorphism
- See code file for examples:
 - Two examples without type variables
 - And one example that does not type-check
 - Then examples for polymorphic functions
 - Nothing changes, just under-constrained: some types can “be anything” but may still need to be the same as other types

Group Work

Write out and solve the constraints for the following programs:

```
let rec f x =  
  match x with  
  | [] -> 0  
  | (a, b) :: tl ->  
    if a then  
      b + f tl  
    else  
      f tl
```

Group Work

Write out and solve the constraints for the following programs:

```
let g x y =  
  match y with  
  | None -> x  
  | Some v -> v  
  
let r = g 1 None
```

Group Work

Write out and solve the constraints for the following programs:

```
let h f x =  
  if x then  
    f 1  
  else  
    f x
```

Two more topics

1. Type-inference story so far is too lenient
 - Value restriction limits where polymorphic types can occur
 - See why and then what
2. OCaml is in a “sweet spot”
 - Type inference more difficult without polymorphism
 - Type inference more difficult with subtyping

Important to “finish the story” but these optional topics are:

- A bit more advanced
- A bit less elegant

The Problem

As described so far, the type system is *unsound* (i.e., *broken*)!

- Allows putting a value of type `t1` (e.g., `int`) where we expect a value of type `t2` (e.g., `string`)

A combination of polymorphism and mutation is to blame:

- Assignment type-checks because (infix) `:=` has type `'a ref -> 'a -> unit`, so instantiate with `string option`
- Dereference type-checks because `!` has type `'a ref -> 'a`, so instantiate with `int option`

```
let r = ref None (* val r : 'a option ref *)
let _ = r := Some "hi"
let i = (Option.get (!r)) / 3
```

What to do

To restore soundness, need a stricter type system that rejects at least one of these three lines

```
let r = ref None (* val r : 'a option ref *)  
let _ = r := Some "hi"  
let i = (Option.get (!r)) / 3
```

- And cannot make special rules for reference types because type-checker cannot know the definition of all type synonyms
 - Due to module system

```
type 'a foo = 'a ref  
let f : 'a -> 'a foo = ref  
let r = f None
```

The fix

```
let r = ref None (* val r : 'weak4 option ref *)  
let _ = r := Some "hi"  
let i = Option.get (!r) / 3
```

- Value restriction: a variable-binding can have a polymorphic type only if the expression is a variable or value
 - Function calls like `ref None` are neither
 - Else OCaml uses a “weak type” to ensure it gets filled in with at most one monomorphic type based on later use
- Not obvious this suffices to make type system sound, but it does

The downside

As we saw previously, the value restriction can cause problems when it is unnecessary because we are not using mutation

```
let pairWithOne = List.map (fun x -> (x,1))  
(* does not get type 'a list -> ('a*int) list *)
```

Type-checker does not know `List.map` is not making a mutable reference

b

Saw a workaround in previous segment on partial application:

```
let pairWithOne xs = List.map (fun x -> (x,1)) xs  
(* 'a list -> ('a*int) list *)
```

If it did make a mutable reference, it would be a new one each time!

A local optimum

- Despite the value restriction, OCaml type inference is elegant and fairly easy to understand
- More difficult *without* polymorphism
 - What type should length-of-list have?
- More difficult *with* subtyping (later course topic)
 - Suppose pairs are supertypes of wider tuples
 - Then `let (y, z) = x` constrains `x` to have at least two fields, not exactly two fields
 - Depending on details, languages can support this, but types often more difficult to infer and understand